

Plant Seedlings Classification with Neural Networks

Damian Franco
Dept. of Computer Science
University of New Mexico
dfranco24@unm.edu

Meiling Traeger
Dept. of Computer Science
University of New Mexico
meilingt@unm.edu

Abstract—This project is intended to teach us about neural networks and their functionality and goals specifically related to image classification. We are given multiple data sets of twelve plant seedling species for training our model and classifying the plant seedling correctly just based off an image. TensorFlow and Keras was the driving forces behind the design and implementation of our model and how we processed our images. We used a combination of feed-forward neural networks and convolutional neural networks as a basis for our model that we trained to classify these plant seedlings. We found that with a combination of both models in a single, sophisticated model, we were able to correctly predict as high as 94% of plant seedlings in our data set just based on images. Our biggest takeaway was that making a model, even with the help from various resources and libraries, is much more than simply calling API functions. There are many hyper-parameters at play in these models, and various ways to correctly and incorrectly implement a successful model for image classification.

Index Terms—image classification, neural networks, big data, machine learning, tensorflow, keras, convolutional neural networks, relu, softmax, L2 regularization, Adam optimizer

[Link to Jupyter notebook on Google Colab.](#)

I. INTRODUCTION

The field of machine learning has seen rapid development in recent years, particularly in the area of neural networks. These networks have been used for a wide range of applications, including image classification, where the goal is to classify images based on their content. In this project, we focused on developing and training a neural network to accurately classify numerous plant seedling species based on their images. We utilized TensorFlow and Keras, two popular machine learning libraries, to create a model that combined both feed-forward and convolutional neural networks. Our results showed that we were able to accurately predict up to 94% of plant seedlings in our dataset. A big factor of correct image classification was tuning the hyperparameters to generate the best implementation model to produce various levels of accuracy. Throughout this paper we will discuss our approach to building this model and share our insights into the challenges of creating a successful model for image classification.

The main objective of this project was to investigate the capabilities and limitations of neural networks, with a specific focus on image classification. To achieve this goal, we used these networks to classify numerous images of plant seedling species. Image recognition is a crucial area of research in machine learning and computer vision, owing to the increasing availability of high-quality image data and advancements in

deep learning algorithms. It has become an important tool for a wide range of applications, including autonomous driving, medical diagnosis, and security systems. Our goal is to gain a deeper understanding of the workings of neural networks and to develop a successful model that can accurately classify the plant seedlings. We hope that this project will provide us with valuable insights into the complexities of building a neural network-based model for image classification. By doing so, we aim to apply these insights to other machine learning projects in the future. Ultimately, we aim to demonstrate the effectiveness of using neural networks in real-world applications and contribute to the growing body of research in this exciting field.

II. METHODS

A feed-forward neural network is a type of neural network where the information flows in only one direction, from input to output. The feed-forward neural network component of our model is responsible for the classification of the plant seedlings. The convolutional neural network component of our model is used to extract relevant features from the images. These features are then passed on to the feed-forward neural network component, which classifies the plant seedlings based on these features.

The use of feed-forward neural networks is particularly important in this project because it allows us to classify the plant seedlings based on their images, without having to manually specify the features that are relevant for the classification task. This is because the network learns to recognize relevant features from the images during the training process. By using feed-forward neural networks in conjunction with convolutional neural networks, we were able to develop a powerful model that accurately classifies plant seedlings based on their images.

Convolutional neural networks (CNNs) are a type of neural network that are particularly well-suited to image recognition tasks, due to their ability to extract relevant features from images. CNNs consist of multiple layers of convolutions, which are operations that apply a filter to an input image to extract features. These features are then passed on to successive layers of the network for further processing. In this project, we used CNNs to extract relevant features from the plant seedling images, which were then fed into a feed-forward neural network for classification.

Residual neural networks are a type of CNN that were introduced to address the problem of vanishing gradients in very deep neural networks. Residual neural networks contain shortcut connections, which allow information to bypass some of the layers in the network. This helps to ensure that the gradients do not become too small as they propagate through the network. Residual neural networks are a type of CNN, as they also use convolutions to extract features from images, but with the added benefit of improved gradient flow.

Keras is a high-level neural network API that is built on top of the Tensorflow library. Tensorflow is an open-source library for numerical computation that is widely used for developing and training machine learning models. In this project, we used Keras and Tensorflow to design and implement our model for classifying plant seedlings based on their images. Keras provides a simple and intuitive interface for building neural network models, which allows us to rapidly prototype and experiment with different network architectures. Tensorflow provides the underlying computational infrastructure for running the neural network computations efficiently.

We used Keras to define the architecture of our neural network, including the number and type of layers, the activation functions, and the loss functions. Keras also provided us with a wide range of optimization algorithms that we could use to train our model, such as stochastic gradient descent and Adam optimizer. Tensorflow allowed us to take advantage of the powerful computational capabilities of GPUs to train our model much faster than would have been possible on a CPU. Additionally, Tensorflow provides tools for visualizing the performance of our model during training, which allowed us to monitor the training process and make adjustments as needed. Overall, the combination of Keras and Tensorflow was essential for the success of our project, allowing us to quickly and effectively develop a powerful model for classifying plant seedlings based on their images.

The flatten layer is a type of layer in neural networks that is used to convert the output of the previous layer into a one-dimensional vector. In convolutional neural networks, the output of the convolutional layers is typically a three-dimensional tensor, where the dimensions correspond to the width, height, and number of channels in the image. The flatten layer is used to convert this tensor into a one-dimensional vector, which can then be fed into a fully connected layer for further processing.

In our project, we used a flatten layer after the last convolutional layer in our model. This allowed us to convert the output of the convolutional layers into a one-dimensional vector, which could then be passed to a feed-forward neural network for classification. The flatten layer ensured that the output of the convolutional layers had the appropriate shape to be processed by the fully connected layers of the feed-forward neural network. Without the flatten layer, the fully connected layers would not have been able to process the output of the convolutional layers, as they require a one-dimensional vector as input. The flatten layer played a crucial role in our model, allowing us to effectively combine the output of the

convolutional layers with the feed-forward layers to achieve high accuracy in classifying the plant seedling images.

In neural networks, activation functions are used to introduce non-linearity into the output of a neuron. Two commonly used activation functions in our project were the Rectified Linear Unit (ReLU) and the Softmax function.

ReLU is a simple activation function that sets any negative input to zero and leaves positive inputs unchanged. ReLU is widely used in convolutional neural networks due to its simplicity and effectiveness. ReLU helps to reduce the likelihood of vanishing gradients during training, which can occur with other activation functions, such as sigmoid or tanh. In our model, we used ReLU as the activation function in the convolutional layers to introduce non-linearity and improve the model's accuracy.

The Softmax function is another commonly used activation function in neural networks, particularly in classification tasks. The Softmax function maps the output of a neural network to a probability distribution over a set of classes. The output of the Softmax function is a set of values between zero and one, where the sum of the values is equal to one. In our project, we used the Softmax function as the activation function in the output layer of our model, which allowed us to calculate the probability of each plant seedling belonging to each of the twelve classes.

The Adam optimizer is a popular optimization algorithm used in deep learning, particularly in neural networks for image classification. Adam stands for Adaptive Moment Estimation and it is an extension of the stochastic gradient descent algorithm.

The Adam optimizer is designed to dynamically adjust the learning rate during training, based on the gradient magnitudes and the variance of the gradient. This allows the model to quickly converge to a good solution and avoid getting stuck in local minima. The Adam optimizer also takes into account the previous steps taken during optimization and computes the adaptive learning rate for each weight of the model, leading to faster and more accurate convergence.

In our project, we used the Adam optimizer to train our model. The Adam optimizer is a widely used optimization algorithm and is available in many deep learning libraries, including TensorFlow and Keras. By using the Adam optimizer, we were able to effectively train our model to classify the plant seedling images with high accuracy, while also avoiding issues such as overfitting and slow convergence.

The cross-entropy loss function is a commonly used loss function in neural networks, particularly in classification tasks. The goal of the cross-entropy loss function is to measure the difference between the predicted probabilities of the model and the actual probabilities of the ground truth labels. The cross-entropy loss function is calculated as the negative log likelihood of the true label given the predicted probabilities.

In our project, we used the cross-entropy loss function to train our model. The cross-entropy loss function is widely used in classification tasks, as it penalizes the model more strongly for incorrect predictions and encourages the model to focus

on the most relevant features of the input data. The cross-entropy loss function, combined with the Softmax activation function in the output layer, allowed us to calculate the loss of our model during training and adjust the model's parameters accordingly to minimize the loss.

A. Our Model Selection:

Our model specifically will be a combination of a convolutional neural network and a feed forward neural network. The model starts with an input layer of size (width=300, height=300, depth=3) which are values that we found that looked the best. The input layer is then passed through an convolutional neural network with an InceptionResNetV2 layer that has pre-trained weights from ImageNet. The output of the InceptionResNetV2 layer is then flattened using a Flatten layer, and passed through a fully connected Dense layer with 128 units, ReLU activation, and L2 regularization. Finally, the output of this layer is passed through another Dense layer with 12 units and a softmax activation function. The model is trained using the categorical cross-entropy loss function and the Adam optimizer with a learning rate of 0.00005 which we found that is the best, and its accuracy is evaluated during training.

III. RESULTS

Our testing process consisted of first constructing a model that was sufficient enough to classify the images of plant seedlings that we were given. We presented the final model that we found and shown in Fig. 1 was the most effective. We tried different variations of just feed-forward neural networks and convolutional neural networks and decided on combining both. The Adam optimizer and L2 regularization techniques were the first and last choices for us to when it came to optimization and regularization techniques. We were taught both during class lectures and stuck with both. The same goes for the activation functions that we used, ReLU for our hidden layer and softmax for our output layer.

A. Data Set Exploration:

Our training data set had a total of 4750 images of plant seedlings. Some plant seedlings had much more images compared to others, as you can see in Fig. 2. Our distribution in our training data set seems to favor the Loose Silky-bent and Common Chickweed. These two species have over 600 images to train on, so we are expecting these two species to have more data to train over. The next notable aspect of our training data is that Black-grass, Cleavers, Common Wheat, Maize and Shepherds Purse all have relatively low number of images being around 300 or less. This distribution is interesting and we are looking forward to seeing if this may have in factor into classification errors for some of these species.

Next, we wanted to view the images themselves and compare them to others to see if there is any distinct similarities between some species with others. In Fig. 3, you can see a comparison of just a couple of images from each species compared to the other species. We can see that there are

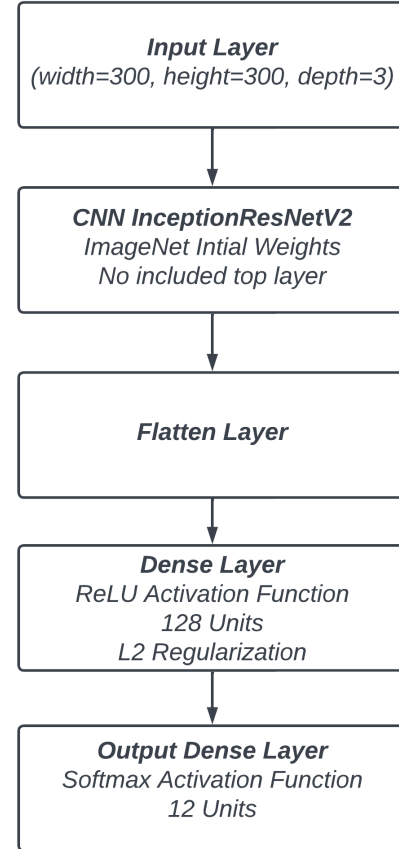


Fig. 1. This figure shows a flow chart of our final model that we designed to correctly classify the plant seedlings. Here, you can see our input, hidden and output layers. Within our first hidden network layer is actually an entire convolutional, residual neural network that gets flattened and passed through a dense layer with a ReLU activation function. We then use a softmax activation function to attempt to classify the plan seedlings.

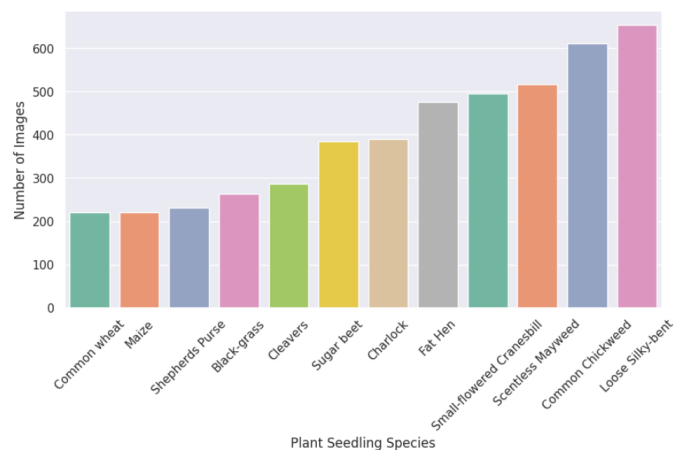


Fig. 2. This figure shows the distribution of our training data set species. There are some species that have many more images to train our model on compared to others. For example, Loose Silky-bent and Maize are polar opposites with the Loose Silky-bent species having more than 400 images more than the Maize species.

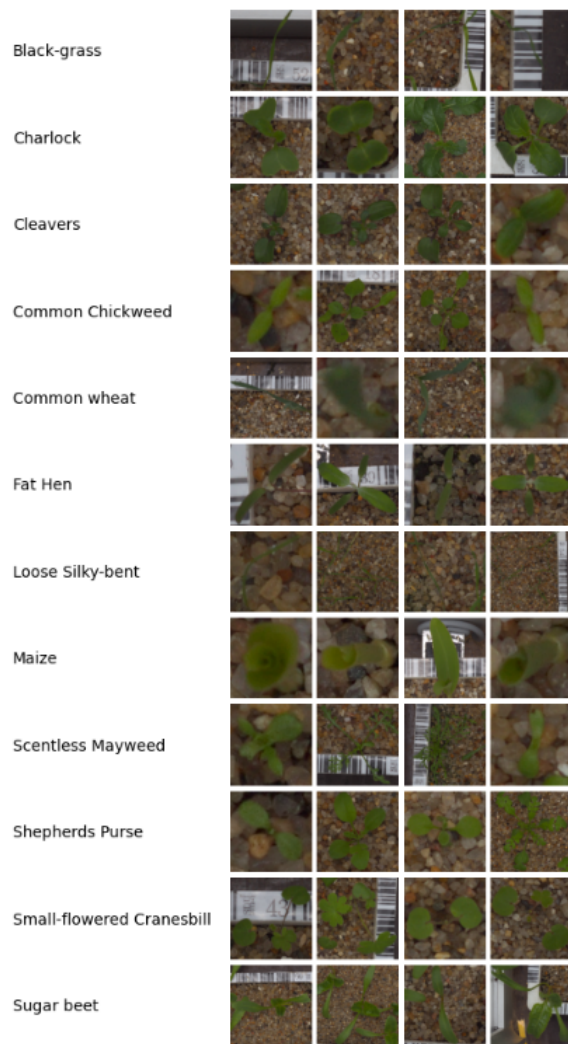


Fig. 3. This figure shows the comparison for our plant seedlings in a single plot. The plot shows a couple of images of our species side by side with all other species so we can view the features of each plant seedlings species and compare them to other species. We noticed that there are plants that have very distinct features compared to other like Maize and some that have similar features compared to other species such as Cleavers and Common Chickweed.

some species with very distinct features like Maize and other that do not have any distinct features that can possibly be misclassified such as Cleavers and Common Chickweed. We are interested to see if these species that share a similar look will be misclassified or if our model will be robust enough to not have many errors.

B. Data Pre-processing:

We first created some generators for our code to before we could test. Within our code, we defined generators for training, validation, and testing data sets. Keras pre-processing was very helpful in generating our data sets for this classification project. The pre-processing library in Keras applies various data augmentation techniques to the training images to improve the model's ability to generalize to new data. We generated three data sets, one for training, one for validation

and the last for testing. Our validation set was of 950 images or 20% of our training data set and our training set is the other 80%. All were generated from the data set .CSV files that were given to us on Kaggle.

In our code, we used various techniques to ensure that our data was properly prepared before feeding it into our model. One important technique we used was normalization, which involves scaling the pixel values of our images to a fixed range. Normalization helps to reduce the effect of lighting variations and improves the convergence speed of our model during training. We also performed data cleaning to remove any corrupted or low-quality images that could negatively impact our model's accuracy. By taking these steps to pre-process our data, we ensured that our model was able to learn the underlying patterns and features of our images more effectively.

Another critical aspect of data pre-processing for image classification is data balancing. In many cases, data sets may have class imbalances, where one or more classes have significantly fewer samples than others. This can lead to bias in the model's predictions, where it may favor the more represented classes over the minority classes. To address this, we used data augmentation techniques to increase the number of samples for the underrepresented classes. This involved applying transformations such as rotation, flipping, and cropping to our training data, effectively creating new samples with slightly different features. By using data augmentation, we were able to balance our data set and improve the overall performance of our model.

The last bit of pre-processing we did was defining a callback. Callbacks are a powerful tool for improving the performance of deep learning models, and they can be particularly useful in image classification tasks. More specifically, they are functions that can be called at specific points during the training process, allowing us to perform various actions or optimizations. In our project, we used a callback function called ModelCheckpoint to save the best-performing model during training. This callback saves the model's weights to a file when it achieves the highest validation accuracy, allowing us to restore the best model later and avoid over-fitting. The ModelCheckpoint callback is defined in our code with a file path where the saved model will be stored, the monitor parameter set to val acc to track validation accuracy, and the save best only parameter set to True to only save the best model. By using this, we can optimize the performance of our image classification models and achieve better results, which we found was very important to our results.

C. Hyper-parameter Tuning:

After creating the necessary data sets and pre-processing the data, we could now continue on to training, validating and testing our model. Most of our testing was done with our model with a combination of different hyper-parameters such as:

- Input layer dimensions (*width, height, depth*)
- Batch size

		Predicted											
		1	2	3	4	5	6	7	8	9	10	11	12
Actual	1	35	0	0	0	6	0	13	0	0	0	0	0
	2	0	61	2	0	0	0	0	0	0	0	0	0
	3	0	0	41	0	0	1	0	0	1	4	0	0
	4	0	0	0	84	0	0	0	0	0	1	0	0
	5	2	0	0	0	38	0	3	0	0	0	0	0
	6	3	0	0	0	2	68	0	0	0	0	0	0
	7	22	0	0	0	1	0	77	0	0	0	0	0
	8	2	0	0	0	0	0	0	39	0	0	0	0
	9	0	0	0	0	0	0	0	0	84	0	0	1
	10	0	2	0	3	0	0	0	0	0	44	0	2
	11	0	0	0	0	1	2	0	0	0	0	65	1
	12	0	0	0	0	0	0	1	0	0	0	0	60

Fig. 4. This figure shows the confusion matrix on our validation data set with our most successful model. We can see in our confusion matrix that there was more misclassification between class 7 (Loose Silky-bent) and class 1 (Black-grass) which are two plant seedlings that look oddly similar. If you take a look at the images, you can see that each of these plant seedlings are very small, with little to no prominent features for each plant seedlings which is why we believe that this is having misclassification errors. Other than that, we seem to have little to no large misclassifications besides those two plant seedlings.

- Number of epochs
- Number of units per layer
- Trained Weights in InceptionResNet
- L2 regularization factor
- Learning rate
- Adam optimizer exponential decay rate

These outline most, if not all of the hyper-parameters that we had to tune for our model. We found that the hyper-parameters that made the most different in improving or decreasing accuracy was the learning rate, number of epochs and number of units per layer. We quickly found that the number of units per layer was never changing when we settled on the correct number of both dense layers at 128 and 12 units. The learning rate would be the next hyper-parameter that was very important which we tested for learning rates from 0.1 to 0.000001 and found that the learning rate 0.00005 was the best and provided the best accuracy for our data sets. Lastly, the number of epochs was the hyper-parameter that influenced our testing and validation accuracy the best. We found as we increased the number of epochs, our model got substantially better, but would over-fit after a large amount of epochs, so we had to find a great number of epochs to get our best score 94%, which was around 4. After 4 epochs, we had significant loss in accuracy.

D. Model Training and Evaluation:

Training our model was fairly simple and took around 30-45 minutes per epoch with 296 steps per epoch to train. After our first epoch, we would stop it training and test out our classification for some runs. Other tests include hyper-parameter tuning and only classifying the validation set and more. As you can see in Fig. 4, our confusion matrix shows one of our best models that for classification and what classes got predicted correctly and what our model was misclassifying. We can see that most of the misclassification appeared in

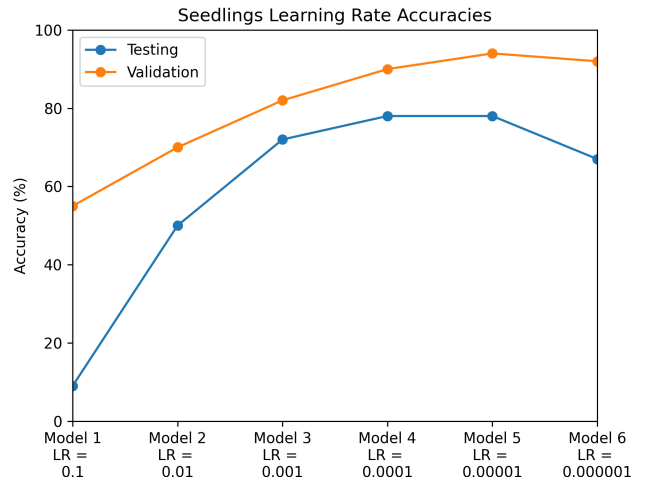


Fig. 5. This figure shows the outcome of both our validation and testing data set when using different learning rate values from 0.1 to 0.000001. We see in that the validation and testing accuracy is always increasing until Model 6 where both seem to trend down with the testing accuracy shooting down at an extreme level. That was a sign of over-fitting that we noticed and why we choose the learning rate of 0.00005.

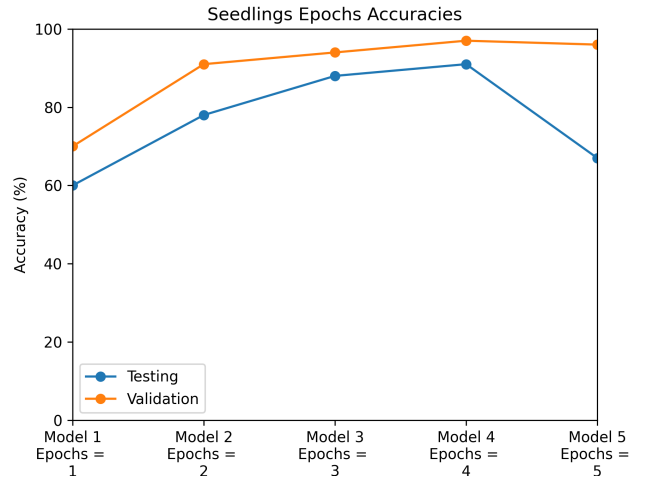


Fig. 6. This figure shows the outcome of both our validation and testing data set when using different epochs from 1 to 5. We see in that the validation and testing accuracy is slightly increasing with some great accuracy with only one epoch. Four epochs was our sweet spot which we tested at around a 90% accuracy for the testing set. The sign of over-fitting is at epoch 5 and it is very apparent that over-fitting is occurring with how low the accuracy is.

images that look very similar and have no prominent features, which is expected.

On to more findings, the figures that show our findings, Fig. 5 and Fig. 6 show two of our case studies that we did for some hyper-parameter tuning and how we found our best run of correctly classifying 94% of the plant seedlings testing data set. The first figure, Fig. 5, shows exactly how we tested different learning rates to find that learning rate was very important to our models accuracy. All models are trained with one epoch and different learning rates and we found that our

best learning rate was 0.00001 and eventually 0.00005 where it was peaking at around 78% in a single epoch which was very impressive to us. The next figure, Fig. 6, is showing different models with a learning rate of 0.00005 training over different number of epochs. We found that with the number of epochs, our model kept getting better until we hit a max of 94% correctly classifying the plant seedlings. After four epochs, we found that we were most likely over-fitting because our accuracy at five epochs was very low compared to the other.

IV. DISCUSSION AND CONCLUSIONS

One of our main takeaways from this project is the process it takes to create an intelligent and robust model for image classification. We were astonished with the amount of options that TensorFlow and Keras have, but not all options are viable or appropriate. Developers have to carefully consider which algorithms and techniques to use based on the nature of the data, the project requirements, and the computational resources available. They also need to fine-tune the hyper-parameters and perform extensive testing to ensure that the model is accurate, efficient, and generalizes well to new data. Another important aspect is data pre-processing and augmentation, which can significantly improve the performance of the model. Overall, building a successful image classification model requires a combination of domain knowledge, experimentation, and computational skills, and it is an iterative process that involves continuous improvement and refinement.

More specifically on experimentation, we noticed that in order to get a model to be successful, a very large amount of experimentation must be done. We did countless experimenting with multiple different models to get a correct model that we thought was sufficient enough to classify the seedling images. After experimenting with models, we then had to do experimentation with different hyper-parameter values to get the best accuracy that we can get. Experimentation takes a great amount of time. Each epoch was about a 30-45 minute training session with our model with another 10-15 minutes to classify our validation and testing data set with our trained model. Time is another aspect of importance. Developers need time to experiment enough times to create a successful model.

Another takeaway that we had was that hyper-parameter tuning is extremely important. We already knew this topic, but it is even more prevalent when using libraries to help us implement our model. Incorrect hyper-parameters can be the catalyst to an entire models way to classify correctly. These hyper-parameters can be labeled as "incorrect" in the slightest sense. As we saw in our learning rate plot and example, we would slightly change our learning rate from 0.00001 to 0.00005 to increase our accuracy by over 5%-10%. We changed other hyper-parameters and saw an accuracy decrease of over 15%. That is validation for us to confidently say that hyper-parameter tuning and choosing correct hyper-parameters is one of the most important part of building machine learning models.

Convolutional Neural Networks and feed-forward Neural Networks are both crucial for image classification problems.

CNNs are specifically designed for processing and classifying images, as they can automatically learn hierarchical representations of image features by applying convolutional filters and pooling layers. This enables them to capture important spatial relationships between pixels and to detect patterns at different scales and orientations. On the other hand, FNNs are more suitable for tasks that require a global perspective on the data, such as recognizing objects or scenes based on high-level features. They are commonly used as the final layers of a CNN, where they can take the flattened outputs of the convolutional layers and map them to the output classes. By combining the strengths of both types of networks, we can create powerful models that can accurately classify a wide range of images, from simple shapes to complex scenes, and adapt to different data distributions and noise levels. A combination of both is exactly what we did for our model and is why we believe we got so much success from this project.

Overall, this project helped us become more informed and develop a great understanding about implementing neural network models. We both learned a tremendous amount through this process and had a great time doing it. Some limitations that we had was due to time constraints. If we had more time to test more models with various different hyper-parameters or larger epoch sizes, then we feel like our model could of easily classified to a high accuracy like 95% to 99% of the testing images. We feel like we had a very good model that could successfully classify plant seedlings. For the future, we are willing to spend more time training the model with the proper amount of epochs to see if our scores improve, or if we will continue to over-fit. If we continue to over-fit, then we can more properly tune some of the hyper-parameters to train a model that could classify better, but we believe the overall structure of the model is great. We are looking forward to continuing to see improvements with future experiments!

REFERENCES

- [1] T. M. Mitchell et al., Machine learning. wcb, 1997.
- [2] Goodfellow, Ian, et al. Deep Learning. The MIT Press, 2017.
- [3] Duch W, Adamczak R, Grabczewski K (1996) Extraction of logical rules from training data using backpropagation networks, in: Proc. of the The 1st Online Workshop on Soft Computing, 19-30.Aug.1996, pp. 25-30, available on-line at: <http://www.bioele.nuee.nagoya-u.ac.jp/wsc1/>